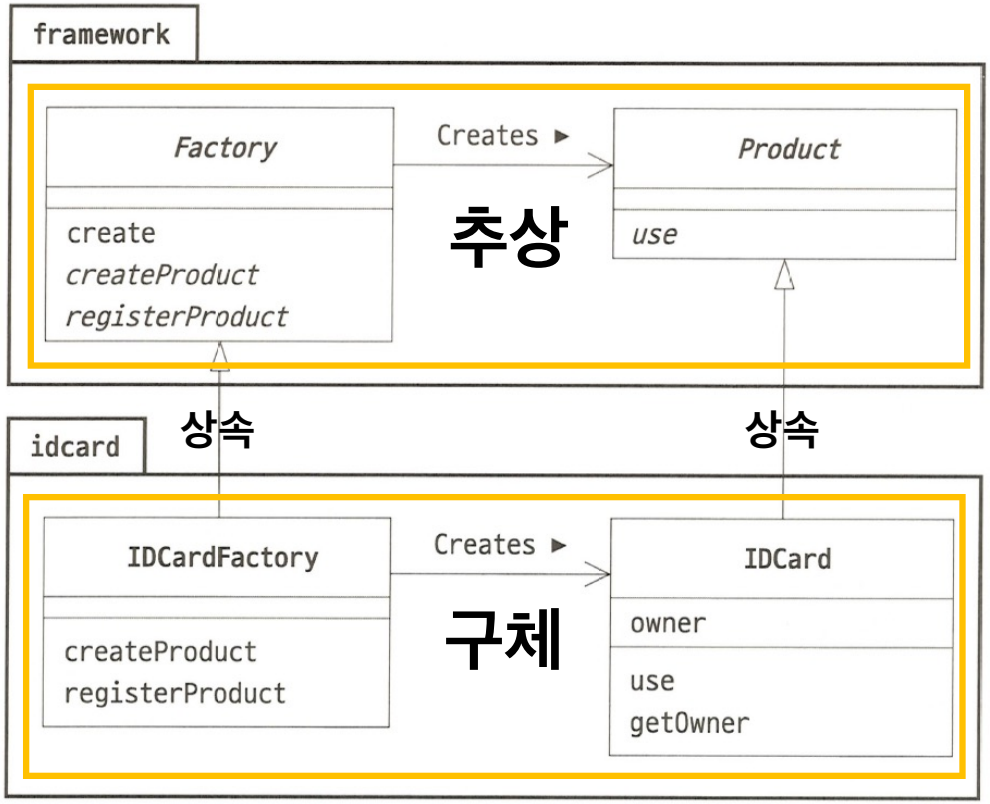
디자인 패턴

2026. 5. 12 w7
미림 마이스터고

남궁성
seong.namkung@gmail.com

17. Factory Method – 자손에서 인스턴스를 만든다.



17. Factory Method – 자손에서 인스턴스를 만든다.

추상

```java
// Product 인터페이스
interface Shape {
    void draw();
}
```

```java
// Creator (Factory)
abstract class ShapeFactory {
    // Factory Method
    public abstract Shape createShape();

    // 공통 로직 처리
    public void renderShape() {
        Shape shape = createShape();
        shape.draw();
    }
}
```

구현

구현

구체

```java
// Concrete Products
class Circle implements Shape {
    @Override
    public void draw() {
        System.out.println("Circle을 그립니다.");
    }
}
```

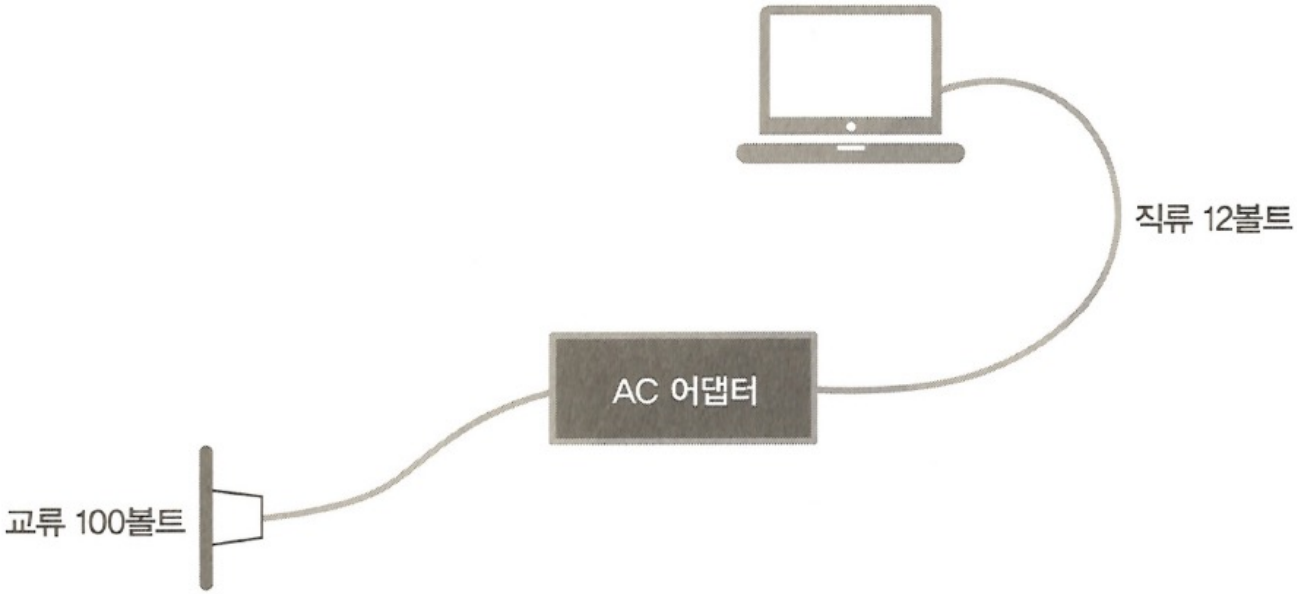
```java
// Concrete Creators
class CircleFactory extends ShapeFactory {
    @Override
    public Shape createShape() {
        return new Circle();
    }
}
```

17. Factory Method – 자손에서 인스턴스를 만든다.

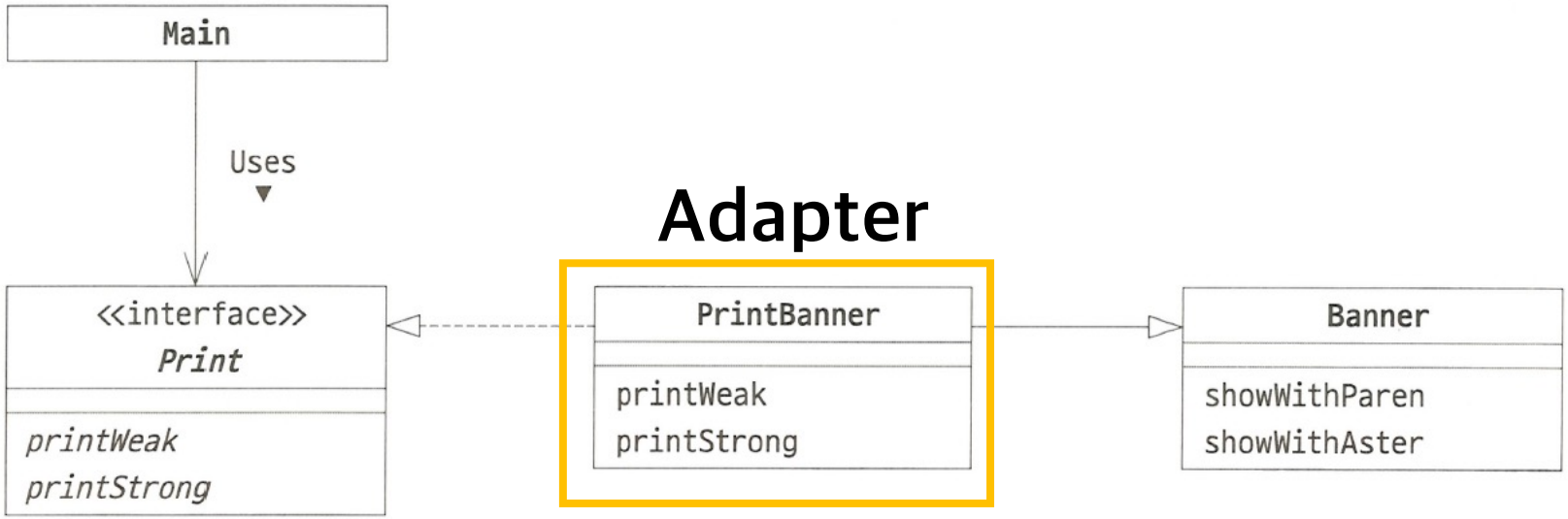
```java
public class FactoryMethodTest2 {
    public static void main(String[] args) {
        ShapeFactory circleFactory = new CircleFactory();
        circleFactory.renderShape();  // Circle을 그립니다.

        ShapeFactory rectangleFactory = new RectangleFactory();
        rectangleFactory.renderShape();  // Rectangle을 그립니다.

        ShapeFactory triangleFactory = new TriangleFactory();
        triangleFactory.renderShape();  // Triangle을 그립니다.
    }
}
```

```
Circle을 그립니다.
Rectangle을 그립니다.
Triangle을 그립니다.
```

18. Adapter – 중간에 끼워서 재사용



18. Adapter – 중간에 끼워서 재사용



18. Adapter – 중간에 끼워서 재사용

```java
public class AdapterTest {
    public static void main(String[] args) {
        Print p = new PrintBanner("Hello");
        p.printWeak();
        p.printStrong();
    }
}
```

사용

```java
interface Print {
    void printWeak();
    void printStrong();
}
```

구현

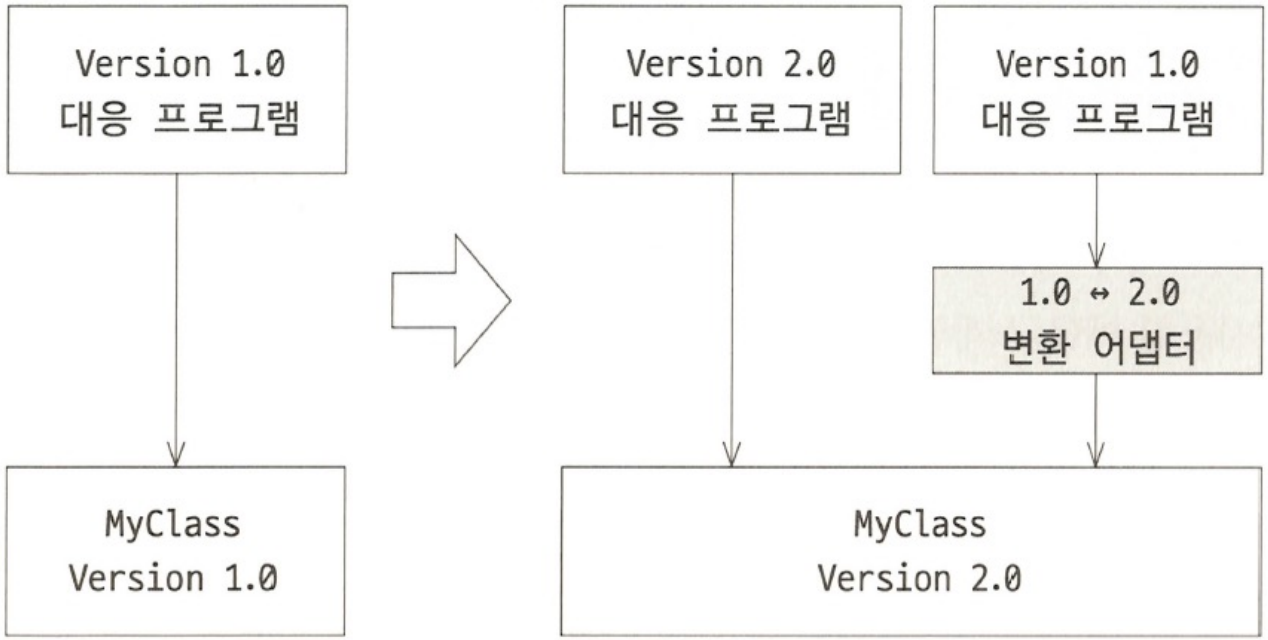
Adapter

```java
class PrintBanner extends Banner implements Print {
    PrintBanner(String string) {
        super(string);
    }

    public void printWeak() {
        showWithParen();
    }
    public void printStrong(){
        showWithAster();
    }
}
```

상속

```java
class Banner {
    private String string;

    Banner(String string){
        this.string = string;
    }
    void showWithParen() {
        System.out.println("("+string+")");
    }
    void showWithAster() {
        System.out.println("*"+string+"*");
    }
}
```

18. Adapter – 중간에 끼워서 재사용



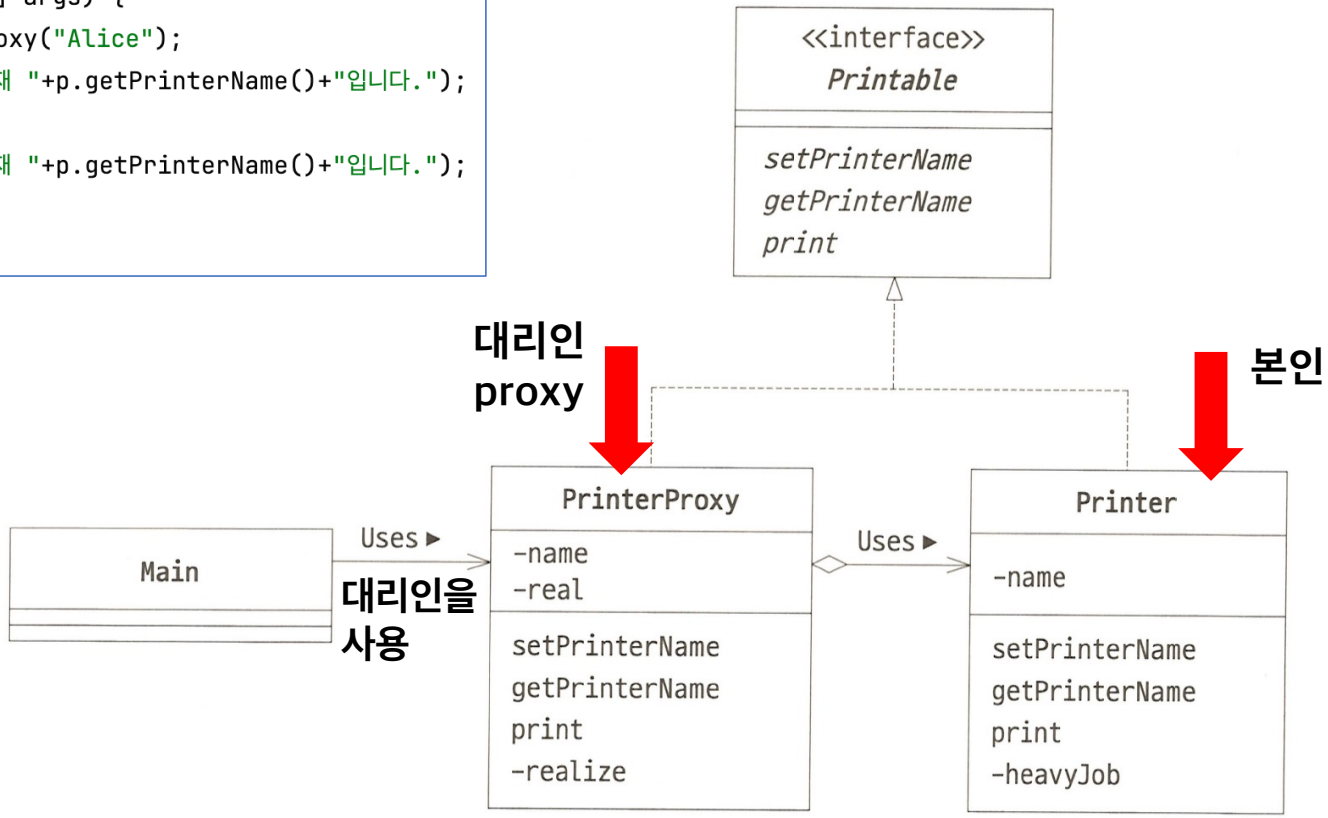
19. Proxy – 대리인(proxy)에게 위임(실제론 본인이 처리)

객체(본인)를 직접 사용하지 않고,
proxy객체(대리인)을 통해 간접적으로 객체(본인)를 사용

1. 객체(본인)을 숨길 수 있다. – 내부 구현을 감출 수 있다.(제어)
2. 객체(본인)의 생성을 늦출 수 있다. – 지연된 생성
3. 부가기능의 추가 – 핵심 로직과 부가 기능의 분리

19. Proxy – 대리인(proxy)에게 위임(실제론 본인이 처리)

```java
public static void main(String[] args) {
    Printable p = new PrinterProxy("Alice");
    System.out.println("이름은 현재 "+p.getPrinterName()+"입니다.");
    p.setPrinterName("Bob");
    System.out.println("이름은 현재 "+p.getPrinterName()+"입니다.");
    p.print("Hello, world");
}
```



19. Proxy – 대리인(proxy)에게 위임(실제론 본인이 처리)

구현

위임(본인에게)

```java
class PrinterProxy implements Printable {
    private String name;
    private Printer real;

    PrinterProxy(String name){
        this.name = name;
        this.real = null;
    }
    public synchronized void setPrinterName(String name){
        if(real != null)
            real.setPrinterName(name);
        this.name = name;
    }
    public String getPrinterName() { return name; }
    public void print(String string) {
        realize();
        real.print(string);
    }
    private synchronized void realize() {
        if(real==null)
            real = new Printer(name);
    }
}
```

```java
interface Printable {
    void setPrinterName(String name);
    String getPrinterName();
    void print(String string);
}
```

```java
class Printer implements Printable {
    private String name;

    Printer(String name){
        this.name = name;
        heavyJob("Printer 인스턴스("+ name +") 생성 중");
    }
    public void setPrinterName(String name) { this.name = name; }
    public String getPrinterName() { return name; }
    public void print(String string) {
        System.out.println("=== " + name + " ===");
        System.out.println(string);
    }
    private void heavyJob(String msg){
        System.out.println(msg);
        for (int i = 0; i < 5; i++) {
            try {
                Thread.sleep(1000);
            } catch(InterruptedException e) {}
            System.out.print(".");
        }
        System.out.println("완료");
    }
}
```